

STOCK TRADING SYSTEM

Database Management Systems



IIT PALAKKAD

Instructor

Koninika Pal

Submitted By

142401038	Jahnavi
142401040	Joshitha
142401024	Greeshma
142401022	Rishitha

Department of Data Science

Contents

1	Abstract	3
2	Introduction	3
3	Problem Statement	3
4	Objectives	3
5	System Overview	4
5.1	Trader	4
5.2	Analyst	4
5.3	Admin	4
6	Database Design	5
6.1	Entity Relationship Diagram	5
6.2	Relational Schema	6
6.2.1	users	6
6.2.2	wallet	6
6.2.3	exchange	6
6.2.4	stocks	6
6.2.5	orders	7
6.2.6	trades	7
6.2.7	transaction	7
6.2.8	portfolio	7
6.2.9	watchlist	7
6.2.10	daily_prices	8
6.3	Relationships	8
7	Integrity Constraints and General Constraints	8
7.1	Primary Key Constraints	8
7.2	Foreign Key Constraints	9
7.3	NOT NULL Constraints	9
7.4	UNIQUE Constraints	9
7.5	CHECK Constraints	9
7.6	DEFAULT Constraints	10
8	Stored Functions and Triggers	10
8.1	register_user()	10
8.2	login_user()	10
8.3	validate_order_inputs() [Helper]	11
8.4	reserve_buy_funds() [Helper]	11
8.5	check_sell_shares() [Helper]	11
8.6	insert_pending_order() [Helper]	11
8.7	match_buy_order() [Helper]	11
8.8	match_sell_order() [Helper]	11
8.9	finalize_order_status() [Helper]	12
8.10	place_order()	12

8.11	cancel_order()	13
8.12	deposit_funds()	14
8.13	get_user_summary()	14
8.14	get_stock_price_history()	15
8.15	get_top_traded_stocks()	15
8.16	get_exchange_stats()	16
8.17	adjust_wallet_balance()	17
9	Triggers	18
9.1	TRG-1: trg_wallet_balance_check	18
9.2	TRG-2: trg_stock_shares_check	19
9.3	TRG-3: trg_trade_updates_price	19
9.4	TRG-4: trg_order_status_cancel	20
9.5	TRG-5: trg_order_status_guard (NEW)	22
10	Roles and Privileges	23
10.1	admin_role	24
10.2	trader_role	24
10.3	analyst_role	24
11	Views	24
11.1	user_portfolio_view	24
11.2	order_summary_view	25
11.3	wallet_transaction_view	26
11.4	stock_market_overview	27
11.5	watchlist_with_prices	28
11.6	open_order_book	28
12	Indexes	29
12.1	Orders Table Indexes	29
12.2	Daily Prices Indexes	32
12.3	Trades Table Indexes	32
12.4	Transaction Table Indexes	33
12.5	Portfolio, Watchlist, and Stocks Indexes	34
13	Implementation Details	36
14	Output	37
15	Conclusion	37

1 Abstract

This report documents the complete design, implementation, and testing of the Stock Trading System, a full-stack database-backed application. The system simulates a real-world limit-order stock exchange, enabling three distinct user roles (Trader, Analyst, and Administrator) to interact with a PostgreSQL relational database through a Flask REST API and a HTML/JavaScript frontend.

Key database achievements include a decomposed modular matching engine implemented as five automated triggers that enforce wallet integrity, share availability, order status life-cycle, and automatic daily-price aggregation, six optimised views that flatten multi-table joins for fast frontend consumption, and seventeen carefully chosen indexes including partial and composite indexes that accelerate the matching engine and analytics queries. The full system encompasses ten relational tables, over fifteen stored functions, five triggers, six views, three roles.

2 Introduction

A stock trading system is one of the most demanding applications for a relational database. It requires strict transactional consistency (ACID guarantees), concurrent access management, complex constraint enforcement, and sub-millisecond query latency on read-heavy market-overview pages, all while maintaining a complete and auditable financial record.

This project builds such a system from scratch using PostgreSQL as the database engine, Flask as the web server, and vanilla HTML/CSS/JavaScript as the frontend. Rather than delegating business logic to the application layer, the system deliberately encodes as much logic as possible inside the database through stored functions, triggers, and views. This design philosophy keeps the backend thin and ensures that all financial operations are atomic, consistent, and reversible regardless of which client calls them.

3 Problem Statement

Traditional brokerage systems face issues like race conditions, double spending, stale price data, and inconsistent order handling.

This project solves these by:

- Ensuring **atomic order execution** (all trade operations happen together).
- Preventing **negative wallet balances**.
- Maintaining a strict **order lifecycle** (PARTIAL/COMPLETED/CANCELLED).
- Automatically updating **daily price data**.
- Enforcing **roles** (admin, trader, analyst).

4 Objectives

- Design a **relational database schema** for stock trading.

- Implement a **limit-order matching engine** using PL/pgSQL.
- Enforce **data integrity using triggers**.
- Create **6 views** for efficient data access.
- Add **17 optimized indexes** for performance.
- Implement roles: **admin, trader, analyst**.
- Build a **Flask API with JWT authentication**.
- Develop a **single-page frontend dashboard**.

5 System Overview

The system supports three user roles, each with a distinct interaction model:

5.1 Trader

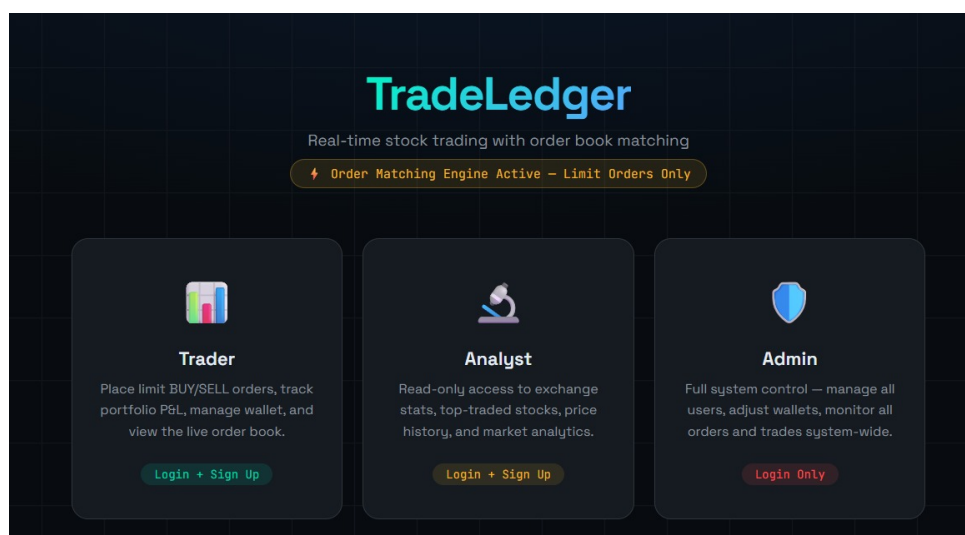
A Trader can register, log in, deposit funds, browse the live market overview, view price history charts, place and cancel limit buy/sell orders, monitor their portfolio with unrealised P&L, manage a personal watchlist, and view wallet transaction history.

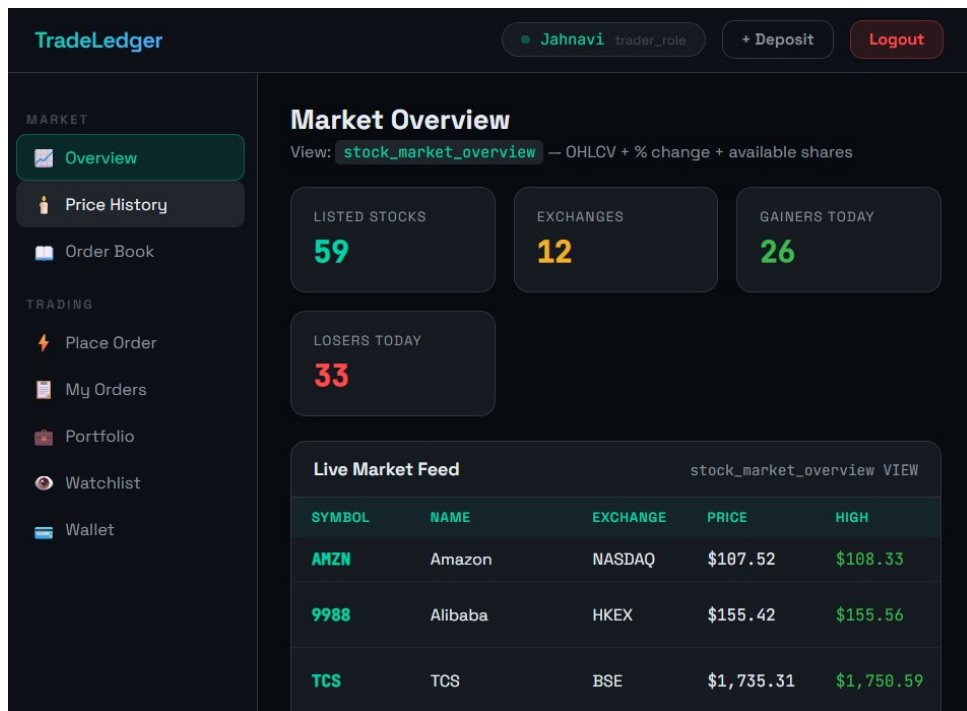
5.2 Analyst

An Analyst is a read-only role with elevated analytics access. Analysts can view the full market overview, study top-traded stocks by volume, examine exchange-level statistics, and view aggregated per-trader profit/loss tables. Analysts cannot place orders or modify any data.

5.3 Admin

The Administrator role has full control over the system. In addition to all analyst capabilities, administrators can view all users and orders, adjust wallet balances, add new stocks with automatic share-count suggestions, and pull a per-user financial summary (wallet balance, portfolio value, order count).

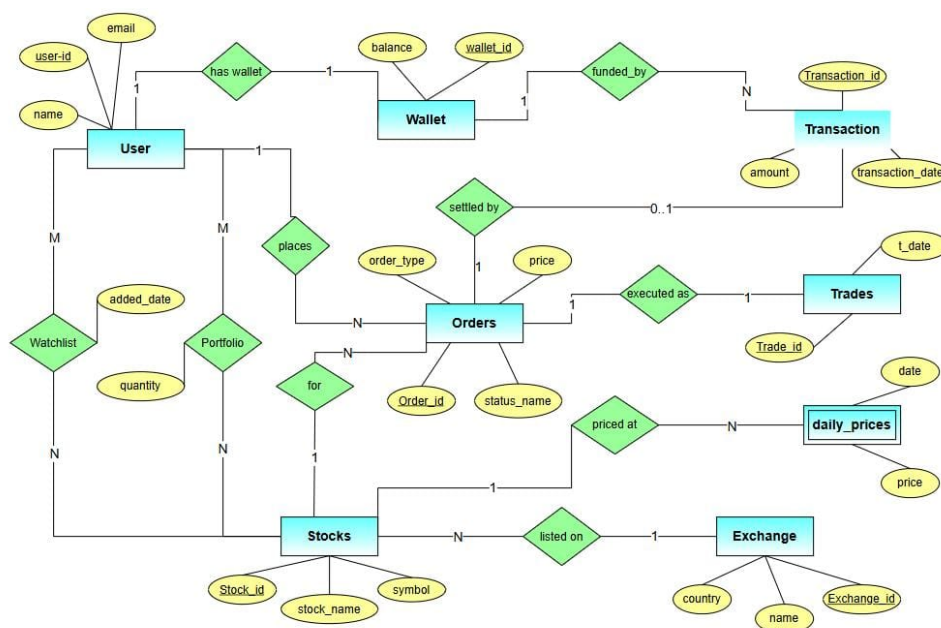




6 Database Design

6.1 Entity Relationship Diagram

The ER diagram below carries forward into the final implementation. It captures all ten entities and their relationships including Users, Wallet, Stocks, Exchange, Orders, Trades, Transaction, Portfolio, Watchlist, and Daily Prices.



6.2 Relational Schema

The full relational model consists of ten tables. All foreign keys enforce referential integrity. Cascading deletes are not used; child rows must be removed explicitly to prevent accidental data loss.

6.2.1 users

Attribute	Data Type	Constraint
user_id	SERIAL	PK, NOT NULL
name	VARCHAR(100)	NOT NULL
email	VARCHAR(150)	NOT NULL, UNIQUE
password	VARCHAR(255)	NOT NULL
phone	VARCHAR(20)	UNIQUE
user_role	VARCHAR(20)	NOT NULL, DEFAULT 'trader', CHECK IN ('admin', 'trader', 'analyst')

6.2.2 wallet

Attribute	Data Type	Constraint
wallet_id	SERIAL	PK, NOT NULL
user_id	INT	NOT NULL, UNIQUE, FK $\rightarrow$ users
balance	DECIMAL(12,2)	NOT NULL, DEFAULT 0, CHECK ≥ 0
last_updated	TIMESTAMP	DEFAULT NOW()

6.2.3 exchange

Attribute	Data Type	Constraint
exchange_id	SERIAL	PK, NOT NULL
name	VARCHAR(100)	NOT NULL
country	VARCHAR(100)	NOT NULL

6.2.4 stocks

Attribute	Data Type	Constraint
stock_id	SERIAL	PK, NOT NULL
stock_name	VARCHAR(100)	NOT NULL
symbol	VARCHAR(10)	NOT NULL, UNIQUE
exchange_id	INT	NOT NULL, FK $\rightarrow$ exchange
total_shares	BIGINT	NOT NULL, CHECK > 0
available_shares	BIGINT	NOT NULL, CHECK $\geq 0, \leq$ total_shares
current_price	DECIMAL(12,2)	nullable

6.2.5 orders

Attribute	Data Type	Constraint
order_id	SERIAL	PK, NOT NULL
user_id	INT	NOT NULL, FK → users
stock_id	INT	NOT NULL, FK → stocks
order_type	VARCHAR(10)	NOT NULL, CHECK IN ('BUY', 'SELL')
quantity	INT	NOT NULL, CHECK > 0
filled_qty	INT	NOT NULL, DEFAULT 0, CHECK ≥ 0
price	DECIMAL(12,2)	NOT NULL, CHECK > 0
order_time	TIMESTAMP	DEFAULT NOW()
status_name	VARCHAR(20)	DEFAULT 'PENDING', CHECK IN ('COMPLETED', 'PENDING', 'CANCELLED', 'PARTIAL')

6.2.6 trades

Attribute	Data Type	Constraint
trade_id	SERIAL	PK, NOT NULL
t_date	DATE	NOT NULL, DEFAULT CURRENT_DATE
buy_order_id	INT	NOT NULL, FK → orders
sell_order_id	INT	NOT NULL, FK → orders
traded_qty	INT	NOT NULL, CHECK > 0
traded_price	DECIMAL(12,2)	NOT NULL

6.2.7 transaction

Attribute	Data Type	Constraint
transaction_id	SERIAL	PK, NOT NULL
wallet_id	INT	NOT NULL, FK → wallet
order_id	INT	nullable, FK → orders
amount	DECIMAL(12,2)	NOT NULL
transaction_date	TIMESTAMP	DEFAULT NOW()
transaction_status	VARCHAR(20)	NOT NULL, CHECK IN ('SUCCESS', 'FAILED', 'REFUNDED')

6.2.8 portfolio

Attribute	Data Type	Constraint
user_id	INT	PK (composite), FK → users
stock_id	INT	PK (composite), FK → stocks
quantity	INT	NOT NULL, CHECK ≥ 0
avg_buy_price	DECIMAL(12,2)	nullable

6.2.9 watchlist

Attribute	Data Type	Constraint
user_id	INT	PK (composite), FK → users
stock_id	INT	PK (composite), FK → stocks
added_date	TIMESTAMP	DEFAULT NOW()

6.2.10 daily_prices

Attribute	Data Type	Constraint
date	DATE	PK (composite)
stock_id	INT	PK (composite), FK → stocks
open_price	NUMERIC	NOT NULL, CHECK > 0
close_price	NUMERIC	NOT NULL, CHECK > 0
high_price	NUMERIC	NOT NULL, CHECK > 0
low_price	NUMERIC	NOT NULL, CHECK > 0
volume	BIGINT	NOT NULL, DEFAULT 0

6.3 Relationships

- **User** → **Wallet** (1:1) — each user has exactly one wallet
- **User** → **Orders** (1:M) — a user can place many orders
- **User** ↔ **Stocks** (M:M) — via Watchlist and Portfolio relationships
- **Wallet** → **Transaction** (1:M) — a wallet funds many transactions
- **Orders** → **Transaction** (1:0..1) — an order may be settled by one transaction
- **Orders** → **Trades** (1:1) — each order is executed as one trade
- **Orders** → **Stocks** (M:1) — orders are placed for a stock
- **Stocks** → **Daily_prices** (1:M) — each stock has many daily price records
- **Stocks** → **Exchange** (M:1) — stocks are listed on an exchange

7 Integrity Constraints and General Constraints

7.1 Primary Key Constraints

Every table has a uniquely identifying primary key. For junction tables (portfolio, watchlist, daily_prices) the primary key is a composite of the two foreign keys, ensuring one record per entity-pair.

Table	Primary Key
users	user_id
wallet	wallet_id
exchange	exchange_id
stocks	stock_id
orders	order_id
trades	trade_id
transaction	transaction_id
portfolio	(user_id, stock_id)
watchlist	(user_id, stock_id)
daily_prices	(date, stock_id)

7.2 Foreign Key Constraints

Table	Foreign Key Column	References
wallet	user_id	users(user_id)
stocks	exchange_id	exchange(exchange_id)
orders	user_id	users(user_id)
orders	stock_id	stocks(stock_id)
trades	buy_order_id	orders(order_id)
trades	sell_order_id	orders(order_id)
transaction	wallet_id	wallet(wallet_id)
transaction	order_id	orders(order_id)
portfolio	user_id, stock_id	users, stocks
watchlist	user_id, stock_id	users, stocks
daily_prices	stock_id	stocks(stock_id)

7.3 NOT NULL Constraints

All financially significant columns are declared NOT NULL to prevent silent null-propagation errors in arithmetic. Key NOT NULL columns include:

- **users:** name, email, password, user_role
- **wallet:** user_id, balance
- **stocks:** stock_name, symbol, exchange_id, total_shares, available_shares
- **orders:** user_id, stock_id, order_type, quantity, filled_qty, price, status_name
- **trades:** t_date, buy_order_id, sell_order_id, traded_qty, traded_price
- **transaction:** wallet_id, amount, transaction_status
- **portfolio:** user_id, stock_id, quantity
- **daily_prices:** date, stock_id, open_price, close_price, high_price, low_price

7.4 UNIQUE Constraints

- **users.email:** Prevents duplicate registrations.
- **users.phone:** Prevents duplicate phone numbers.
- **wallet.user_id:** Enforces one wallet per user.
- **stocks.symbol:** Ticker symbols must be globally unique.
- **portfolio(user_id, stock_id):** Enforced by the composite primary key; one portfolio row per user-stock pair.
- **watchlist(user_id, stock_id):** Enforced by the composite primary key; duplicate watchlist entries are rejected.

7.5 CHECK Constraints

- **wallet.balance CHECK (balance >= 0):** Enforced at the column level and also by the `trg_prevent_negative_balance` trigger for double safety.
- **stocks.total_shares CHECK (total_shares > 0):** IPO supply must be positive.

- `stocks.available_shares` CHECK (`available_shares >= 0`): Cannot go negative; also enforced by trigger.
- `stocks` CHECK (`available_shares <= total_shares`): Available cannot exceed total supply.
- `orders.order_type` CHECK IN (`'BUY', 'SELL'`): Only valid order directions accepted.
- `orders.quantity / filled_qty` CHECK `> 0 / >= 0`: Quantity must be positive; filled starts at zero.
- `orders.status_name` CHECK IN (`'COMPLETED', 'PENDING', 'CANCELLED', 'PARTIAL'`): Only valid lifecycle states accepted.
- `transaction.transaction_status` CHECK IN (`'SUCCESS', 'FAILED', 'REFUNDED'`): Only valid financial outcomes.
- `users.user_role` CHECK IN (`'admin', 'trader', 'analyst'`): Only valid system roles.

7.6 DEFAULT Constraints

- `wallet.balance` DEFAULT `0.00`: New traders start with zero balance.
- `orders.filled_qty` DEFAULT `0`: Every order starts unfilled.
- `orders.status_name` DEFAULT `'PENDING'`: All orders begin in the pending state.
- Transaction timestamps and order timestamps DEFAULT `NOW()`: Automatic audit trail.

8 Stored Functions and Triggers

This section documents every stored function and trigger in the system. Functions are written in PL/pgSQL and compiled inside PostgreSQL. They run atomically inside the calling transaction.

8.1 `register_user()`

Description

Handles new user registration for the trader and analyst roles. Administrator accounts are managed outside the database via environment variables to prevent privilege escalation through the sign-up endpoint.

8.2 `login_user()`

Description

Authenticates a user by verifying the supplied password against the stored bcrypt hash. Returns a status of “success”, “invalid” (wrong password), or “not_found” (email not in the system) so the Flask layer can return the appropriate HTTP status code.

8.3 `validate_order_inputs()` [Helper]

Description

A lightweight helper function that serves as the first gate in the order-placement pipeline. It raises an exception for any invalid input before any row lock or wallet debit occurs, keeping error messages close to the user-supplied data.

8.4 `reserve_buy_funds()` [Helper]

Description

Handles all pre-order checks and fund reservation for BUY orders. This function acquires row-level locks on both the stock and wallet rows to prevent race conditions, then debits the full order cost upfront (escrow model).

8.5 `check_sell_shares()` [Helper]

Description

Pre-order checks for SELL orders. Verifies that the user holds at least `p_qty` shares in their portfolio before allowing the order to proceed.

8.6 `insert_pending_order()` [Helper]

Description

Inserts the new order row with status `PENDING` and returns its `order_id`. For BUY orders, it also links the upfront wallet debit (already recorded by `reserve_buy_funds`) to this specific `order_id`, closing an audit-trail gap that existed in the earlier monolithic version.

8.7 `match_buy_order()` [Helper]

Description

The core matching engine for BUY orders. Iterates over eligible SELL orders in price-ascending, time-ascending order (best price first, FIFO for ties) and executes as many trades as possible until the BUY order is fully filled or no matching SELL orders remain.

8.8 `match_sell_order()` [Helper]

Description

The mirror of `match_buy_order` for SELL-initiated orders. Iterates over eligible BUY orders in price-descending, time-ascending order (best price first, FIFO for ties). Key difference: `available_shares` is NOT decremented here because shares move between two portfolios in the secondary market; the stock pool was already adjusted when the matching BUY was reserved.

8.9 finalize_order_status() [Helper]

Description

Sets the conclusive status on the submitted order after the matching engine completes. This function is the final step in the `place_order` orchestrator and ensures the order's status correctly reflects how much was filled.

8.10 place_order()

```

1 CREATE OR REPLACE FUNCTION place_order(
2   p_user_id INT,
3   p_stock_id INT,
4   p_type VARCHAR,
5   p_qty INT,
6   p_price DECIMAL
7 )
8 RETURNS INT
9 LANGUAGE plpgsql AS $$
10 DECLARE
11   v_order_id INT;
12   v_wallet_id INT;
13   v_total_cost NUMERIC := p_qty * p_price;
14 BEGIN
15   PERFORM validate_order_inputs(p_type, p_qty, p_price);
16
17   IF p_type = 'BUY' THEN
18     v_wallet_id := reserve_buy_funds(
19       p_user_id, p_stock_id, p_qty, p_price, v_total_cost
20     );
21   ELSIF p_type = 'SELL' THEN
22     v_wallet_id := check_sell_shares(p_user_id, p_stock_id, p_qty);
23   END IF;
24
25   v_order_id := insert_pending_order(
26     p_user_id, p_stock_id, p_type, p_qty, p_price,
27     v_wallet_id, v_total_cost
28   );
29
30   IF p_type = 'BUY' THEN
31     PERFORM match_buy_order(
32       v_order_id, p_stock_id, p_user_id, v_wallet_id, p_price, p_qty
33     );
34   ELSIF p_type = 'SELL' THEN
35     PERFORM match_sell_order(
36       v_order_id, p_stock_id, p_user_id, v_wallet_id, p_price, p_qty
37     );
38   END IF;
39
40   PERFORM finalize_order_status(v_order_id);
41

```

```

42 RETURN v_order_id;
43 END;
44 $$;
45
46 GRANT EXECUTE ON FUNCTION place_order(INT, INT, VARCHAR, INT,
    DECIMAL)
47 TO trader_role, admin_role;

```

Listing 1: place_order() – Main Orchestrator

Description

The main entry point for order submission. This function is a clean orchestrator that delegates every concern to a focused helper, making the code easy to reason about and test in isolation. The full end-to-end flow runs inside a single PostgreSQL transaction.

8.11 cancel_order()

```

1 CREATE OR REPLACE FUNCTION cancel_order(p_order_id INT,
    p_user_id INT)
2 RETURNS TEXT
3 LANGUAGE plpgsql AS $$
4 DECLARE
5     v_order RECORD;
6     v_wallet_id INT;
7     v_unfilled INT;
8     v_refund NUMERIC;
9 BEGIN
10     SELECT * INTO v_order FROM orders
11     WHERE order_id = p_order_id FOR UPDATE;
12
13     IF NOT FOUND THEN
14         RAISE EXCEPTION 'Order % not found', p_order_id;
15     END IF;
16
17     IF v_order.user_id <> p_user_id THEN
18         RAISE EXCEPTION 'You can only cancel your own orders';
19     END IF;
20
21     IF v_order.status_name = 'COMPLETED' THEN
22         RAISE EXCEPTION 'Completed orders cannot be cancelled';
23     END IF;
24
25     IF v_order.status_name = 'CANCELLED' THEN
26         RAISE EXCEPTION 'Order is already cancelled';
27     END IF;
28
29     v_unfilled := v_order.quantity - v_order.filled_qty;
30
31     IF v_order.order_type = 'BUY' AND v_unfilled > 0 THEN

```

```

32 SELECT wallet_id INTO v_wallet_id FROM wallet WHERE user_id =
    p_user_id;
33 v_refund := v_unfilled * v_order.price;
34
35 UPDATE wallet
36 SET balance = balance + v_refund, last_updated = NOW()
37 WHERE wallet_id = v_wallet_id;
38
39 INSERT INTO transaction(wallet_id, order_id, amount,
    transaction_status)
40 VALUES (v_wallet_id, p_order_id, v_refund, 'REFUNDED');
41 END IF;
42
43 UPDATE orders SET status_name = 'CANCELLED' WHERE order_id =
    p_order_id;
44
45 RETURN 'Order ' || p_order_id || ' cancelled. Unfilled qty: '
    || v_unfilled;
46 END; $$;
47
48 GRANT EXECUTE ON FUNCTION cancel_order(INT,INT) TO trader_role,
    admin_role;

```

Listing 2: cancel_order() – Order Cancellation with Refund

Description

Cancels a PENDING or PARTIAL order placed by the specified user. For BUY orders with unfilled quantity, refunds the reserved wallet funds. SELL orders have no upfront wallet deduction, so no refund is issued. The function inserts the REFUNDED transaction BEFORE updating the order status to prevent a double-refund from the trg_cancel_order_refund safety-net trigger.

8.12 deposit_funds()

Description

Adds funds to a trader’s wallet and records the deposit as a financial transaction. Returns the new wallet balance.

8.13 get_user_summary()

```

1 CREATE OR REPLACE FUNCTION get_user_summary(p_user_id INT)
2 RETURNS TABLE(
3   user_name TEXT,
4   wallet_balance NUMERIC,
5   portfolio_value NUMERIC,
6   total_orders BIGINT,
7   completed_orders BIGINT
8 )

```

```

9  LANGUAGE plpgsql SECURITY DEFINER AS $$
10 BEGIN
11     RETURN QUERY
12     SELECT
13         u.name::TEXT,
14         COALESCE(w.balance, 0),
15         COALESCE(SUM(p.quantity * dp.close_price), 0)::NUMERIC,
16         COUNT(o.order_id),
17         COUNT(o.order_id) FILTER (WHERE o.status_name = 'COMPLETED')
18     FROM users u
19     LEFT JOIN wallet w ON w.user_id = u.user_id
20     LEFT JOIN orders o ON o.user_id = u.user_id
21     LEFT JOIN portfolio p ON p.user_id = u.user_id
22     LEFT JOIN (
23         SELECT DISTINCT ON (stock_id) stock_id, close_price
24         FROM daily_prices ORDER BY stock_id, date DESC
25     ) dp ON dp.stock_id = p.stock_id
26     WHERE u.user_id = p.user_id
27     GROUP BY u.name, w.balance;
28 END; $$;
29
30 GRANT EXECUTE ON FUNCTION get_user_summary(INT) TO admin_role;

```

Listing 3: get_user_summary() – Admin Financial Snapshot

Description

An admin-facing aggregate function that returns a financial snapshot for any user. Marked `SECURITY DEFINER` so it executes with the function owner’s privileges, allowing the admin role to access view data without granting the underlying table `SELECT` to all callers.

8.14 get_stock_price_history()

Description

Returns the OHLCV price history for a stock for the last `p_days` trading days. A key fix in the final version anchors the date range to the maximum date available in the table (rather than `CURRENT_DATE`) so that seed data ending in December 2024 still returns results when the system is run in 2025 or 2026.

8.15 get_top_traded_stocks()

```

1  CREATE OR REPLACE FUNCTION get_top_traded_stocks(p_n INT DEFAULT
    10)
2  RETURNS TABLE(
3      symbol TEXT,
4      stock_name TEXT,
5      total_trades BIGINT,
6      total_volume_shares BIGINT,
7      total_volume_usd NUMERIC

```



```

8 )
9 LANGUAGE plpgsql AS $$
10 BEGIN
11     RETURN QUERY
12     SELECT
13         s.symbol::TEXT,
14         s.stock_name::TEXT,
15         COUNT(t.trade_id),
16         SUM(t.traded_qty)::BIGINT,
17         ROUND(SUM(t.traded_qty * t.traded_price)::NUMERIC, 2)
18     FROM trades t
19     JOIN orders bo ON bo.order_id = t.buy_order_id
20     JOIN stocks s ON s.stock_id = bo.stock_id
21     GROUP BY s.symbol, s.stock_name
22     ORDER BY total_volume_usd DESC
23     LIMIT p_n;
24 END; $$;
25
26 GRANT EXECUTE ON FUNCTION get_top_traded_stocks(INT) TO
    analyst_role, admin_role;

```

Listing 4: get_top_traded_stocks() – Top Stocks by Volume

Description

Returns the top `p_n` stocks ranked by total traded USD volume. Used by the Analyst dashboard to identify the most actively traded securities.

8.16 get_exchange_stats()

```

1 CREATE OR REPLACE FUNCTION get_exchange_stats()
2 RETURNS TABLE(
3     exchange_name TEXT,
4     country TEXT,
5     num_stocks BIGINT,
6     num_trades BIGINT,
7     total_traded NUMERIC
8 )
9 LANGUAGE plpgsql AS $$
10 BEGIN
11     RETURN QUERY
12     SELECT
13         e.name::TEXT,
14         e.country::TEXT,
15         COUNT(DISTINCT s.stock_id),
16         COUNT(t.trade_id),
17         ROUND(COALESCE(SUM(t.traded_qty * t.traded_price), 0)::NUMERIC,
18             2)
19     FROM exchange e
20     LEFT JOIN stocks s ON s.exchange_id = e.exchange_id
21     LEFT JOIN orders bo ON bo.stock_id = s.stock_id

```

```

21 LEFT JOIN trades t ON t.buy_order_id = bo.order_id
22 GROUP BY e.name, e.country
23 ORDER BY total_traded DESC;
24 END; $$;
25
26 GRANT EXECUTE ON FUNCTION get_exchange_stats() TO analyst_role,
    admin_role;

```

Listing 5: get_exchange_stats() – Exchange Level Summary

Description

Provides an exchange-level summary for the Analyst dashboard, showing how many stocks are listed on each exchange and the total trading volume per exchange.

8.17 adjust_wallet_balance()

```

1 CREATE OR REPLACE FUNCTION adjust_wallet_balance(
2   p_user_id INT,
3   p_amount NUMERIC,
4   p_reason TEXT DEFAULT 'Admin adjustment'
5 )
6 RETURNS NUMERIC
7 LANGUAGE plpgsql SECURITY DEFINER AS $$
8 DECLARE
9   v_new_balance NUMERIC;
10  v_wallet_id INT;
11 BEGIN
12   UPDATE wallet
13   SET balance = balance + p_amount, last_updated = NOW()
14   WHERE user_id = p_user_id
15   RETURNING balance, wallet_id INTO v_new_balance, v_wallet_id;
16
17   IF NOT FOUND THEN
18     RAISE EXCEPTION 'User % not found', p_user_id;
19   END IF;
20
21   INSERT INTO transaction(wallet_id, order_id, amount,
22     transaction_status)
23   VALUES (v_wallet_id, NULL, p_amount, 'SUCCESS');
24
25   RETURN v_new_balance;
26 END; $$;
27
28 GRANT EXECUTE ON FUNCTION adjust_wallet_balance(INT,NUMERIC,TEXT
29   ) TO admin_role;

```

Listing 6: adjust_wallet_balance() – Admin Wallet Adjustment

Description

An admin-only function for making manual wallet adjustments (e.g., bonus credits, fee deductions, or data-fix operations). `SECURITY DEFINER` ensures it runs with elevated privileges.

9 Triggers

Five triggers enforce database-level integrity rules that cannot be violated regardless of which function, role, or direct SQL statement causes the change. Together they form an immutable safety net below the application layer.

9.1 TRG-1: `trg_wallet_balance_check`

Trigger Definition

```
1 CREATE OR REPLACE FUNCTION trg_prevent_negative_balance()  
2 RETURNS TRIGGER LANGUAGE plpgsql AS $$  
3 BEGIN  
4   IF NEW.balance < 0 THEN  
5     RAISE EXCEPTION  
6     'Wallet balance cannot go negative. Current: %, Attempted: %',  
7     OLD.balance, NEW.balance;  
8   END IF;  
9   RETURN NEW;  
10 END; $$;  
11  
12 CREATE TRIGGER trg_wallet_balance_check  
13 BEFORE UPDATE ON wallet  
14 FOR EACH ROW EXECUTE FUNCTION trg_prevent_negative_balance();
```

Listing 7: TRG-1: Prevent Negative Wallet Balance

Purpose

Prevents any operation from reducing a wallet balance below zero. This is a `BEFORE` trigger so it fires before the write is committed, allowing the exception to roll back the entire transaction without leaving corrupt data.

Logic

If `NEW.balance < 0`, raises an `EXCEPTION` with the current and attempted balance values. Returns `NEW` otherwise.

Consistency Role

Guarantees the `balance >= 0` `CHECK` constraint can never be bypassed by a direct `UPDATE` or a future function that forgets to check. Acts as the last line of defence for the wallet invariant.

9.2 TRG-2: trg_stock_shares_check

Trigger Definition

```
1 CREATE OR REPLACE FUNCTION trg_check_available_shares()
2 RETURNS TRIGGER LANGUAGE plpgsql AS $$
3 BEGIN
4     IF NEW.available_shares < 0 THEN
5         RAISE EXCEPTION
6             'Not enough shares available. Stock: %, Tried to reduce by too
              much.',
7             NEW.stock_id;
8     END IF;
9     RETURN NEW;
10 END; $$;
11
12 CREATE TRIGGER trg_stock_shares_check
13 BEFORE UPDATE ON stocks
14 FOR EACH ROW EXECUTE FUNCTION trg_check_available_shares();
```

Listing 8: TRG-2: Prevent Negative Available Shares

Purpose

Prevents the matching engine or any direct SQL from reducing `available_shares` below zero, which would represent selling non-existent shares.

Logic

If `NEW.available_shares < 0`, raises an `EXCEPTION` naming the stock. Returns `NEW` otherwise.

Consistency Role

Provides a second layer of protection beyond the `CHECK` constraint, ensuring stock supply is conserved under concurrent load.

9.3 TRG-3: trg_trade_updates_price

```
1 CREATE OR REPLACE FUNCTION trg_update_daily_volume()
2 RETURNS TRIGGER LANGUAGE plpgsql AS $$
3 DECLARE
4     v_stock_id INT;
5 BEGIN
6     SELECT stock_id INTO v_stock_id
7     FROM orders WHERE order_id = NEW.buy_order_id;
8
9     IF v_stock_id IS NULL THEN
10        RAISE WARNING 'trg_update_daily_volume: could not find stock_id
11            for buy_order_id=%', NEW.buy_order_id;
12    RETURN NEW;
```

```

13  END IF;
14
15  INSERT INTO daily_prices(date, stock_id, open_price,
16      close_price,
17      high_price, low_price, volume)
18  VALUES (
19      NEW.t_date,
20      v_stock_id,
21      NEW.traded_price,
22      NEW.traded_price,
23      NEW.traded_price,
24      NEW.traded_qty
25  )
26  ON CONFLICT (date, stock_id) DO UPDATE
27  SET
28      close_price = NEW.traded_price,
29      high_price = GREATEST(daily_prices.high_price, NEW.traded_price
30      ),
31      low_price = LEAST (daily_prices.low_price, NEW.traded_price),
32      volume = daily_prices.volume + NEW.traded_qty;
33
34  RETURN NEW;
35
36  END; $$;
37
38  CREATE TRIGGER trg_trade_updates_price
39  AFTER INSERT ON trades
40  FOR EACH ROW EXECUTE FUNCTION trg_update_daily_volume();

```

Listing 9: TRG-3: Auto-Update Daily Prices on Trade Insert

Purpose

Every time a trade is recorded, this trigger automatically upserts the record for that day and stock. This eliminates the need for any application-layer price aggregation code.

Logic

Resolves the `stock_id` from the `buy_order_id`. Runs `INSERT INTO daily_prices ... ON CONFLICT (date, stock_id) DO UPDATE`. On conflict, `open_price` is intentionally left unchanged (first-trade-of-day semantics), while `close_price` is set to the latest `traded_price`, `high_price = GREATEST(existing, new)`, `low_price = LEAST(existing, new)`, and volume accumulates additively.

Consistency Role

Keeps `daily_prices` continuously up-to-date without any scheduled job. The `stock_market_overview` view always reflects live data.

9.4 TRG-4: `trg_order_status_cancel`

```
1 CREATE OR REPLACE FUNCTION trg_cancel_order_refund()
2 RETURNS TRIGGER LANGUAGE plpgsql AS $$
3 DECLARE
4     v_wallet_id INT;
5     v_unfilled INT;
6     v_refund NUMERIC;
7     v_already_refunded BOOLEAN;
8 BEGIN
9     IF NEW.status_name <> 'CANCELLED'
10    OR OLD.status_name NOT IN ('PENDING', 'PARTIAL') THEN
11     RETURN NEW;
12 END IF;
13
14 IF OLD.order_type <> 'BUY' THEN
15     RETURN NEW;
16 END IF;
17
18 v_unfilled := OLD.quantity - OLD.filled_qty;
19
20 IF v_unfilled <= 0 THEN
21     RETURN NEW;
22 END IF;
23
24 SELECT EXISTS(
25     SELECT 1 FROM transaction
26     WHERE order_id = OLD.order_id
27     AND transaction_status = 'REFUNDED'
28 ) INTO v_already_refunded;
29
30 IF v_already_refunded THEN
31     RETURN NEW;
32 END IF;
33
34 SELECT wallet_id INTO v_wallet_id
35 FROM wallet WHERE user_id = OLD.user_id;
36
37 IF v_wallet_id IS NULL THEN
38     RAISE WARNING 'trg_cancel_order_refund: no wallet for user_id=%',
39         OLD.user_id;
40     RETURN NEW;
41 END IF;
42
43 v_refund := v_unfilled * OLD.price;
44
45 UPDATE wallet
46 SET balance = balance + v_refund,
47     last_updated = NOW()
48 WHERE wallet_id = v_wallet_id;
49
```

```

50  INSERT INTO transaction(wallet_id, order_id, amount,
    transaction_status)
51  VALUES (v_wallet_id, OLD.order_id, v_refund, 'REFUNDED');
52
53  RETURN NEW;
54  END; $$;
55
56  CREATE TRIGGER trg_order_status_cancel
57  AFTER UPDATE OF status_name ON orders
58  FOR EACH ROW EXECUTE FUNCTION trg_cancel_order_refund();

```

Listing 10: TRG-4: Safety Net Refund on Admin Cancellation

Purpose

Issues a wallet refund for cancelled BUY orders when an administrator directly sets `status_name = 'CANCELLED'` via a raw SQL UPDATE, bypassing the `cancel_order()` function. This trigger is explicitly a SAFETY NET, not the primary cancellation path.

Logic

- Only fires when `NEW.status_name = 'CANCELLED'` and `OLD.status_name IN ('PENDING', 'PARTIAL')`.
- Only acts on BUY orders (SELL orders have no upfront wallet deduction).
- Checks whether a REFUNDED transaction already exists for this `order_id`. If it does, `cancel_order()` already ran — the trigger exits immediately to prevent a double-refund.
- If no REFUNDED row exists (admin direct UPDATE path), credits the wallet and inserts a REFUNDED transaction.

Consistency Role

Ensures BUY order cancellations are always refunded exactly once, regardless of whether they go through `cancel_order()` or a direct admin UPDATE.

9.5 TRG-5: trg_order_status_guard (NEW)

```

1  CREATE OR REPLACE FUNCTION trg_guard_order_status()
2  RETURNS TRIGGER LANGUAGE plpgsql AS $$
3  BEGIN
4    IF NEW.status_name = OLD.status_name THEN
5      RETURN NEW;
6    END IF;
7
8    IF OLD.status_name IN ('COMPLETED', 'CANCELLED') THEN
9      RAISE EXCEPTION
10     'Order % is % and cannot be moved to %. Terminal states are
       immutable.',
11     OLD.order_id, OLD.status_name, NEW.status_name;

```

```
12  END IF;
13
14  IF OLD.status_name = 'PENDING'
15  AND NEW.status_name NOT IN ('PARTIAL', 'COMPLETED', 'CANCELLED'
16  ) THEN
17  RAISE EXCEPTION
18  'Invalid order status transition: % -> % for order %',
19  OLD.status_name, NEW.status_name, OLD.order_id;
20  END IF;
21
22  IF OLD.status_name = 'PARTIAL'
23  AND NEW.status_name NOT IN ('COMPLETED', 'CANCELLED') THEN
24  RAISE EXCEPTION
25  'Invalid order status transition: % -> % for order %',
26  OLD.status_name, NEW.status_name, OLD.order_id;
27  END IF;
28
29  RETURN NEW;
30
31  END; $$;
32
33  CREATE TRIGGER trg_order_status_guard
34  BEFORE UPDATE OF status_name ON orders
35  FOR EACH ROW EXECUTE FUNCTION trg_guard_order_status();
```

Listing 11: TRG-5: Order Lifecycle State Machine Guard

Purpose

Enforces the order lifecycle state machine at the database level. No application bug or direct SQL can place an order into an impossible state (e.g., moving a COMPLETED order back to PENDING).

Consistency Role

Prevents a class of subtle data-corruption bugs where an order might be double-matched or double-refunded due to incorrect status transitions. Provides a compile-time-like contract on order lifecycle.

10 Roles and Privileges

Three database roles implement Role-Based Access Control (RBAC). Each role's privilege set is scoped to the minimum required for its function, following the principle of least privilege.

10.1 admin_role

Privilege Type	Details
Table Access	SELECT, INSERT, UPDATE, DELETE on ALL tables in schema public
Function Access	EXECUTE on ALL functions in schema public
Sequence Access	USAGE, SELECT on ALL sequences in schema public
Exclusive Functions	get_user_summary(), adjust_wallet_balance()

Purpose: Reserved for system administrators who manage users, stocks, exchanges, and monitor all trading activity. Admin credentials are stored in environment variables, not in the users table, to prevent privilege escalation through the sign-up endpoint.

10.2 trader_role

Privilege Type	Details
Table Read	SELECT on ALL tables in schema public
orders	INSERT; UPDATE on filled_qty, status_name columns only
wallet	UPDATE on balance, last_updated columns only
users	UPDATE on name, email, password, phone columns only
portfolio	INSERT, UPDATE, DELETE
watchlist	INSERT, UPDATE, DELETE
transaction	INSERT
stocks	UPDATE on available_shares column only
Sequence Access	USAGE, SELECT on ALL sequences

Purpose: Assigned to end users (traders) who need to interact with the trading system. The column-level UPDATE grants on orders and wallet ensure that a trader can drive the matching engine but cannot change an order's price or arbitrarily set wallet balance.

10.3 analyst_role

Privilege Type	Details
Table Access	SELECT on ALL tables in schema public (read-only)
Function Access	EXECUTE on get_stock_price_history(), get_top_traded_stocks(), get_exchange_stats()
View Access	SELECT on all views (granted explicitly)
No Write Access	Cannot INSERT, UPDATE, or DELETE any data

Purpose: Assigned to analysts who need comprehensive read access and analytics capabilities but must not be able to modify any trading or financial data.

11 Views

Six views are created to flatten complex multi-table joins into single queryable objects. Every view is granted to the appropriate roles and consumed directly by Flask API routes, keeping backend code short and the query plan stable.

11.1 user_portfolio_view

```
1 CREATE OR REPLACE VIEW user_portfolio_view AS
2 SELECT
3     u.user_id,
4     u.name AS user_name,
5     s.stock_id,
6     s.symbol,
7     s.stock_name,
8     p.quantity,
9     COALESCE(p.avg_buy_price, 0) AS avg_buy_price,
10    dp.close_price AS current_price,
11    ROUND((p.quantity * dp.close_price)::NUMERIC, 2) AS
        market_value
12 FROM portfolio p
13 JOIN users u ON u.user_id = p.user_id
14 JOIN stocks s ON s.stock_id = p.stock_id
15 JOIN (
16     SELECT DISTINCT ON (stock_id)
17         stock_id, close_price
18     FROM daily_prices
19     ORDER BY stock_id, date DESC
20 ) dp ON dp.stock_id = p.stock_id
21 WHERE p.quantity > 0;
```

Listing 12: user_portfolio_view – Portfolio with Market Value

Justification

The portfolio page in the frontend requires all these derived columns. Computing them inline in every route would require duplicating a complex three-table join. The view centralises this logic, ensures consistent P&L calculation, and benefits from the `idx_portfolio_user_stock` partial index on live holdings.

Granted To

trader_role, analyst_role, admin_role.

11.2 order_summary_view

```
1 CREATE OR REPLACE VIEW order_summary_view AS
2 SELECT
3     o.order_id,
4     u.user_id,
5     u.name AS user_name,
6     s.symbol,
7     s.stock_name,
8     e.name AS exchange_name,
9     o.order_type,
10    o.quantity,
11    o.filled_qty,
12    o.quantity - o.filled_qty AS remaining_qty,
```

```

13  o.price,
14  ROUND((o.filled_qty * o.price)::NUMERIC, 2) AS filled_value,
15  o.order_time,
16  o.status_name
17  FROM orders o
18  JOIN users u ON u.user_id = o.user_id
19  JOIN stocks s ON s.stock_id = o.stock_id
20  JOIN exchange e ON e.exchange_id = s.exchange_id;

```

Listing 13: order_summary_view – Orders with Stock and Exchange Info

Justification

The admin “all orders” page and the trader “my orders” page both need to show stock symbol and exchange name alongside the order record. This view avoids repeating the three-way join in each endpoint.

Granted To

trader_role, analyst_role, admin_role.

11.3 wallet_transaction_view

```

1  CREATE OR REPLACE VIEW wallet_transaction_view AS
2  SELECT
3    u.user_id,
4    u.name AS user_name,
5    w.wallet_id,
6    w.balance AS current_balance,
7    t.transaction_id,
8    t.amount,
9    t.transaction_date,
10   t.transaction_status,
11   o.order_type,
12   s.symbol
13  FROM wallet w
14  JOIN users u ON u.user_id = w.user_id
15  LEFT JOIN transaction t ON t.wallet_id = w.wallet_id
16  LEFT JOIN orders o ON o.order_id = t.order_id
17  LEFT JOIN stocks s ON s.stock_id = o.stock_id;

```

Listing 14: wallet_transaction_view – Transaction History with Context

Justification

The wallet page must display a transaction history with context (which stock was traded). LEFT JOINS preserve deposit rows where order_id is NULL, which would be lost in an INNER JOIN.

Granted To

trader_role, admin_role.

11.4 stock_market_overview

```
1 CREATE OR REPLACE VIEW stock_market_overview AS
2 WITH ranked AS (
3     SELECT
4         stock_id,
5         open_price,
6         close_price,
7         high_price,
8         low_price,
9         volume,
10        date,
11        ROW_NUMBER() OVER (PARTITION BY stock_id ORDER BY date DESC) AS
12            rn
13    FROM daily_prices
14 )
15 SELECT
16     s.stock_id,
17     s.symbol,
18     s.stock_name,
19     e.name AS exchange_name,
20     e.country,
21     s.total_shares,
22     s.available_shares,
23     r1.open_price,
24     r1.close_price AS latest_price,
25     r1.high_price,
26     r1.low_price,
27     r1.volume,
28     r2.close_price AS prev_close,
29     CASE
30         WHEN r2.close_price IS NULL OR r2.close_price = 0 THEN NULL
31         ELSE ROUND(
32             ((r1.close_price - r2.close_price) / r2.close_price * 100)::
33             NUMERIC,
34             2
35         )
36     END AS pct_change
37 FROM stocks s
38 JOIN exchange e ON e.exchange_id = s.exchange_id
39 LEFT JOIN ranked r1 ON r1.stock_id = s.stock_id AND r1.rn = 1
40 LEFT JOIN ranked r2 ON r2.stock_id = s.stock_id AND r2.rn = 2;
```

Listing 15: stock_market_overview – Live Market Data with Percentage Change

Justification

The market overview page is the most-read page in the system. Encapsulating the percentage-change calculation in a view ensures consistent display across all clients and allows the database planner to optimise the CTE materialisation.

Granted To

trader_role, analyst_role, admin_role.

11.5 watchlist_with_prices

```
1 CREATE OR REPLACE VIEW watchlist_with_prices AS
2 SELECT
3     wl.user_id,
4     u.name AS user_name,
5     s.stock_id,
6     s.symbol,
7     s.stock_name,
8     wl.added_date,
9     dp.close_price AS latest_price,
10    dp.date AS price_date
11 FROM watchlist wl
12 JOIN users u ON u.user_id = wl.user_id
13 JOIN stocks s ON s.stock_id = wl.stock_id
14 LEFT JOIN (
15     SELECT DISTINCT ON (stock_id) stock_id, close_price, date
16     FROM daily_prices ORDER BY stock_id, date DESC
17 ) dp ON dp.stock_id = wl.stock_id;
```

Listing 16: watchlist_with_prices – Watchlist with Latest Prices

Justification

The watchlist page should always show all watched stocks even if price data is absent. The LEFT JOIN preserves all watchlist rows.

Granted To

trader_role, analyst_role, admin_role.

11.6 open_order_book

```
1 CREATE OR REPLACE VIEW open_order_book AS
2 SELECT
3     s.symbol,
4     s.stock_name,
5     o.order_type,
6     o.price,
7     SUM(o.quantity - o.filled_qty) AS open_quantity,
```

```

8  COUNT(*) AS num_orders
9  FROM orders o
10 JOIN stocks s ON s.stock_id = o.stock_id
11 WHERE o.status_name IN ('PENDING', 'PARTIAL')
12 GROUP BY s.symbol, s.stock_name, o.order_type, o.price
13 ORDER BY s.symbol, o.order_type,
14          CASE o.order_type WHEN 'BUY' THEN o.price END DESC,
15          CASE o.order_type WHEN 'SELL' THEN o.price END ASC;

```

Listing 17: open_order_book – Live Order Book Depth

Justification

The order book depth view is a real-time indicator of market liquidity. It directly supports the matching engine by showing traders the available counterparties. The CASE-based ORDER BY correctly interleaves BUY and SELL sides in a single result set.

Granted To

trader_role, analyst_role, admin_role.

12 Indexes

Seventeen non-default indexes are created in addition to the primary-key and unique-constraint indexes automatically generated by PostgreSQL. They are grouped by purpose below.

12.1 Orders Table Indexes

Index Name	Type	Purpose / Query It Supports
idx_orders_user_id	BTREE(user_id)	Trader “my orders” page: WHERE user_id = ‘___’
idx_orders_stock_id	BTREE(stock_id)	Order book page: WHERE stock_id = ‘___’
idx_orders_order_time	BTREE(order_time DESC)	Admin recent-orders list: ORDER BY order_time DESC
idx_orders_status	BTREE(status_name)	Cancel/match: WHERE status_name IN (‘PENDING’, ‘PARTIAL’)
idx_orders_type	BTREE(order_type)	Matching engine filter: WHERE order_type = ‘SELL’
idx_orders_user_stock	BTREE(user_id, stock_id)	Per-user per-stock order history for cancel_order() and order page

Index Name	Type	Purpose / Query It Supports
idx_orders_book	Partial BTREE(stock_id, order_type, status_name, price, order_time) WHERE status IN (‘PENDING’, ‘PARTIAL’)	CRITICAL: Covers the entire matching engine FOR loop scan, keeping the index small by excluding terminal orders
idx_orders_stock_- status_time	Partial BTREE(stock_id, status_name, order_time ASC) WHERE status IN (‘PENDING’, ‘PARTIAL’)	Satisfies the ORDER BY in the matching engine without a filesort

Sample Query Using idx_orders_user_id

```

1 SELECT order_id, stock_id, order_type, quantity, price,
   status_name, order_time
2 FROM orders
3 WHERE user_id = 42
4 ORDER BY order_time DESC;
```

Sample Query Using idx_orders_stock_id

```

1 SELECT order_id, order_type, price, quantity, filled_qty,
   status_name
2 FROM orders
3 WHERE stock_id = 5
4 ORDER BY price ASC;
```

Sample Query Using idx_orders_order_time

```

1 SELECT order_id, user_id, stock_id, order_type, quantity, price,
   status_name, order_time
2 FROM orders
3 ORDER BY order_time DESC
4 LIMIT 50;
```

Sample Query Using idx_orders_status

```

1 SELECT status_name, COUNT(*) AS total
2 FROM orders
```

```
3 WHERE status_name IN ('PENDING', 'PARTIAL')
4 GROUP BY status_name;
```

Sample Query Using idx_orders_type

```
1 SELECT order_id, user_id, price, quantity - filled_qty AS
   open_qty
2 FROM orders
3 WHERE order_type = 'SELL'
4 AND status_name IN ('PENDING', 'PARTIAL');
```

Sample Query Using idx_orders_user_stock

```
1 SELECT order_id, order_type, quantity, filled_qty, price,
   status_name, order_time
2 FROM orders
3 WHERE user_id = 42
4 AND stock_id = 5
5 ORDER BY order_time DESC;
```

Sample Query Using idx_orders_book

```
1 SELECT order_id, user_id, quantity - filled_qty AS open_qty,
   price
2 FROM orders
3 WHERE stock_id = 5
4 AND order_type = 'SELL'
5 AND status_name IN ('PENDING', 'PARTIAL')
6 AND price <= 350.00
7 ORDER BY price ASC, order_time ASC;
```

Sample Query Using idx_orders_stock_status_time

```
1 SELECT order_id, order_type, price, quantity - filled_qty AS
   open_qty
2 FROM orders
3 WHERE stock_id = 5
4 AND status_name IN ('PENDING', 'PARTIAL')
5 ORDER BY order_time ASC;
```


12.2 Daily Prices Indexes

Index Name	Type	Purpose
idx_daily_prices_-stock_date	BTREE(stock_id, date DESC)	get_stock_price_history(): WHERE stock_id = ? ORDER BY date DESC
idx_daily_prices_date	BTREE(date DESC)	Gainers/losers analytics: GROUP BY date for most recent day

Sample Query Using idx_daily_prices_stock_date

```

1 SELECT date, open_price, high_price, low_price, close_price,
   volume
2 FROM daily_prices
3 WHERE stock_id = 5
4 ORDER BY date DESC
5 LIMIT 30;

```

Sample Query Using idx_daily_prices_date

```

1 SELECT stock_id, open_price, close_price, volume
2 FROM daily_prices
3 WHERE date = (SELECT MAX(date) FROM daily_prices)
4 ORDER BY (close_price - open_price) DESC;

```

12.3 Trades Table Indexes

Index Name	Type	Purpose
idx_trades_buy_order	BTREE(buy_order_id)	JOIN trades ON buy_order_id in get_top_traded_stocks()
idx_trades_sell_order	BTREE(sell_order_id)	JOIN trades ON sell_order_id for P&L analytics
idx_trades_date	BTREE(t_date DESC)	Recent trade history: ORDER BY t_date DESC
idx_trades_stock_date	BTREE(t_date DESC)	Daily volume aggregation for gainers/losers view

Sample Query Using idx_trades_buy_order

```

1 SELECT bo.stock_id, SUM(t.traded_qty) AS total_volume
2 FROM trades t
3 JOIN orders bo ON bo.order_id = t.buy_order_id
4 GROUP BY bo.stock_id
5 ORDER BY total_volume DESC
6 LIMIT 10;

```

Sample Query Using idx_trades_sell_order

```

1 SELECT so.user_id, SUM(t.traded_qty * t.traded_price) AS
   total_proceeds
2 FROM trades t
3 JOIN orders so ON so.order_id = t.sell_order_id
4 GROUP BY so.user_id
5 ORDER BY total_proceeds DESC;

```

Sample Query Using idx_trades_date

```

1 SELECT trade_id, t_date, buy_order_id, sell_order_id, traded_qty
   , traded_price
2 FROM trades
3 ORDER BY t_date DESC
4 LIMIT 20;

```

Sample Query Using idx_trades_stock_date

```

1 SELECT bo.stock_id, SUM(t.traded_qty) AS daily_volume
2 FROM trades t
3 JOIN orders bo ON bo.order_id = t.buy_order_id
4 WHERE t.t_date = (SELECT MAX(t_date) FROM trades)
5 GROUP BY bo.stock_id
6 ORDER BY daily_volume DESC;

```

12.4 Transaction Table Indexes

Index Name	Type	Purpose
idx_transaction_-wallet_id	BTREE(wallet_id)	wallet_transaction_view JOIN: WHERE wallet_id = ?
idx_transaction_-order_id	BTREE(order_id)	Link reservation transaction in insert_pending_order()
idx_transaction_status	Partial BTREE(transaction_status) WHERE status IN ('SUCCESS', 'REFUNDED', 'FAILED')	Analytics queries counting by status; trigger guard EXISTS check

Sample Query Using idx_transaction_wallet_id

```

1 SELECT transaction_id, order_id, amount, transaction_date,
   transaction_status
2 FROM transaction
3 WHERE wallet_id = 7
4 ORDER BY transaction_date DESC;

```

Sample Query Using idx_transaction_order_id

```

1 SELECT transaction_id, wallet_id, amount, transaction_date,
   transaction_status
2 FROM transaction
3 WHERE order_id = 101
4 ORDER BY transaction_date ASC;

```

Sample Query Using idx_transaction_status

```

1 SELECT EXISTS (
2   SELECT 1
3   FROM transaction
4   WHERE order_id = 101
5   AND transaction_status = 'REFUNDED'
6 ) AS already_refunded;

```

12.5 Portfolio, Watchlist, and Stocks Indexes

Index Name	Type	Purpose
idx_portfolio_user_id	BTREE(user_id)	Portfolio page: WHERE user_id = ?
idx_portfolio_user_stock	Partial UNIQUE(user_id, stock_id) WHERE quantity > 0	Matching engine INSERT ON CONFLICT; partial keeps index tight to live holdings only
idx_watchlist_user_id	BTREE(user_id)	Watchlist page: WHERE user_id = ?
idx_watchlist_user_stock	UNIQUE(user_id, stock_id)	Watchlist add: ON CONFLICT DO NOTHING membership check
idx_stocks_exchange	BTREE(exchange_id)	Exchange stats JOIN: WHERE exchange_id = ?
idx_stocks_symbol	UNIQUE(symbol)	Symbol-based lookups in every market overview and watchlist-add query

Sample Query Using idx_portfolio_user_id

```

1 SELECT stock_id, quantity, avg_buy_price
2 FROM portfolio
3 WHERE user_id = 42
4 AND quantity > 0;

```

Sample Query Using idx_portfolio_user_stock

```
1 INSERT INTO portfolio (user_id, stock_id, quantity,
   avg_buy_price)
2 VALUES (42, 5, 10, 340.00)
3 ON CONFLICT (user_id, stock_id)
4 DO UPDATE SET
5   quantity = portfolio.quantity + 10,
6   avg_buy_price = ROUND(
7     ((portfolio.quantity * portfolio.avg_buy_price) + (10 * 340.00)
8     )
9   / (portfolio.quantity + 10), 2
10  );
```

Sample Query Using idx_watchlist_user_id

```
1 SELECT stock_id, added_date
2 FROM watchlist
3 WHERE user_id = 42
4 ORDER BY added_date DESC;
```

Sample Query Using idx_watchlist_user_stock

```
1 INSERT INTO watchlist (user_id, stock_id, added_date)
2 VALUES (42, 7, NOW())
3 ON CONFLICT (user_id, stock_id)
4 DO NOTHING;
```

Sample Query Using idx_stocks_exchange

```
1 SELECT stock_id, symbol, stock_name, available_shares,
   current_price
2 FROM stocks
3 WHERE exchange_id = 2
4 ORDER BY symbol ASC;
```

Sample Query Using idx_stocks_symbol

```
1 SELECT stock_id, stock_name, exchange_id, available_shares,
   current_price
2 FROM stocks
3 WHERE symbol = 'INFY';
```

13 Implementation Details

The system is implemented as a full-stack application with the following components working together:

- **Trader Portfolio Page:** Displays live portfolio values, unrealised P&L, and stock holdings using the `user_portfolio_view`.
- **Analyst Dashboard:** Shows top traded stocks by volume and exchange-level statistics using `get_top_traded_stocks()` and `get_exchange_stats()`.
- **Admin Dashboard:** Provides user financial summaries and full order visibility using `get_user_summary()` and `order_summary_view`.

The screenshot shows the 'User Summary' page for a user named 'greeny'. The interface includes a sidebar with navigation options like Price History, Order Book, Place Order, My Orders, Portfolio, Watchlist, Wallet, Gainers & Losers, Trader P&L, Top Stocks, Exchanges, Add Stock, User Summary (selected), All Users, and All Orders. The main content area displays the user's details: NAME (greeny), BALANCE (\$121,685.65), PORTFOLIO VALUE (\$51,689.68), TOTAL ORDERS (6), and COMPLETED ORDERS (2). At the top right, there are links for Administrator, Deposit, and Logout.

The screenshot shows the 'Top Traded Stocks' page. The sidebar is similar to the previous one, but the 'Top Stocks' option is selected. The main content area displays a table of top traded stocks by volume. The table has columns for RANK, SYMBOL, NAME, TRADES, SHARES TRADED, and VOLUME USD. The data is as follows:

RANK	SYMBOL	NAME	TRADES	SHARES TRADED	VOLUME USD
1	META	Meta Platforms	12	255	\$354,819.30
2	UNH	UnitedHealth	12	358	\$241,873.90
3	SHOP	Shopify	13	358	\$27,595.85
4	TCS	TCS	15	680	\$1,032,382.80
5	NFLX	Netflix	15	358	\$250,498.25
6	RY	Royal Bank Canada	18	548	\$53,781.48
7	KO	Coca-Cola	11	468	\$66,921.28
8	CRM	Salesforce	14	325	\$74,468.78
9	TM	Toyota Motor	18	648	\$25,478.35
10	HSBC	HSBC Holdings	15	368	\$28,843.45

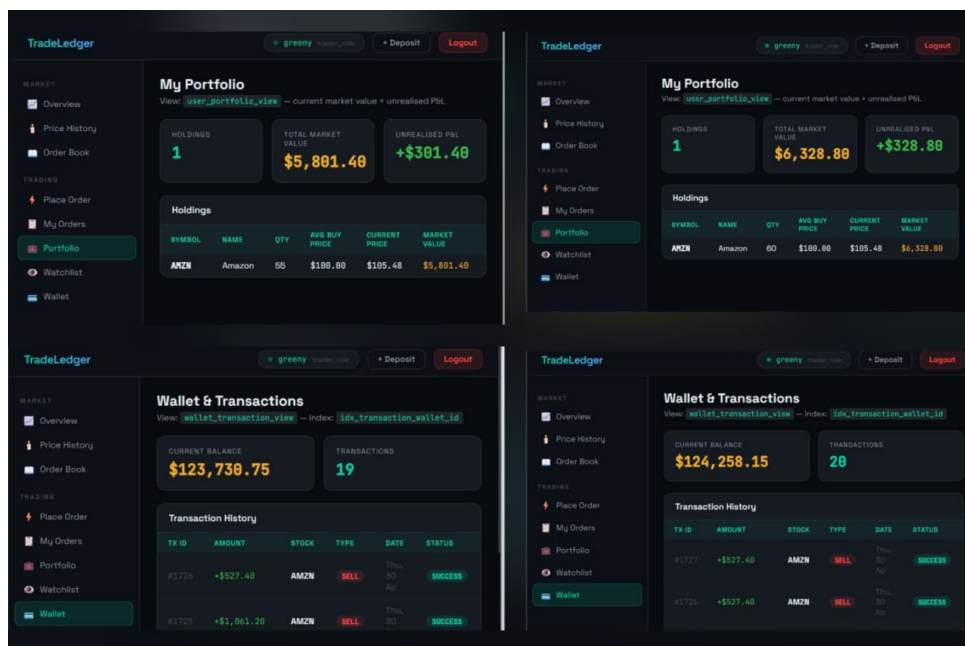
The screenshot shows the 'My Portfolio' page for a user named 'Jahnavi'. The sidebar is similar to the previous ones, but the 'Portfolio' option is selected. The main content area displays the user's portfolio details: HOLDINGS (1), TOTAL MARKET VALUE (\$2,158.48), and UNREALISED P&L (+\$150.48). Below this, there is a table of holdings:

SYMBOL	NAME	QTY	AVG BUY PRICE	CURRENT PRICE	MARKET VALUE	UNREALISED P&L	P&L %
AMZN	Amazon	20	\$189.88	\$187.52	\$2,158.48	+\$150.48	+7.82%

14 Output

The completed system clearly demonstrates all the important database management concepts in a practical way:

- **Strong reliability (ACID properties):** All order-matching operations run inside a single transaction in PostgreSQL. If anything goes wrong, the entire operation is rolled back, so no partial or incorrect data is saved.
- **Safe concurrent operations:** Row-level locking using `FOR UPDATE` ensures that when multiple users place orders at the same time, there are no conflicts or data inconsistencies.
- **Automatic price updates:** The `daily_prices` table is updated automatically through a trigger, ensuring accurate tracking of open, high, low, close, and volume (OHLCV) values in real time.
- **Good performance:** A special index (`idx_orders_book`) is used to speed up the matching process. It only includes active orders (PENDING or PARTIAL), which keeps it efficient and fast.



15 Conclusion

This Stock Trading System proves the use of advanced database technology within an actual life-critical system. Instead of moving all the functionality to the application, in the project business rules are implemented within the database, with the help of such tools as stored functions and triggers. Such an approach ensures reliability and integrity of the data regardless of any user that may modify it.

Another significant advantage of this project lies in its structure. All functions of the trading system are divided into several components, which provides modularity, readability and transactional safety. Besides, there is a group of triggers consisting of five members, which is intended to ensure transactional safety and enforce important business rules.

There are six views utilized within the system, allowing separation of presentation logic from physical data organization. Moreover, seventeen indexes are used to provide better performance of critical queries.